

Command A Crew Of AI Agents

[TUTORIAL](#)[HOW-TO GUIDE](#)[EXPLANATION](#)

A quick overview of multi-agent teams using Ollama and CrewAI

AUTHOR

Edward Jackson

PUBLISHED

January 11, 2024

In this project, we use open-source large language models locally, delegating them to agents responsible for tasks to achieve an overall goal.

We ran our code using Python version **3.11.7** on a Mac M1 chip. You will need the following libraries in your Python environment:

requirements.txt

crewai==0.1.24

langchain==0.0.354

unstructured==0.11.8

Ollama

Ollama provides some really powerful, open-source large-language models (LLMs) such as Mistral, Llama 2 and OpenHermes. The purposes of their models range from chat functionality to code generation, including multiple hybrid models. We will be loading models from Ollama for use locally in this project.

To set up Ollama, simply visit [Ollama](#) and select your operating system. Note that - at the time of writing - Windows is not yet available.

For each model you wish to use locally, you must download them explicitly in your preferred command line (CL). For the mistral model, we type `ollama pull mistral`. Note that these tend to be large files so they may take some time to download, depending on your internet connection.

Next, you can dive straight into Ollama by typing `ollama run mistral`. You can replace **mistral** with any of the other models available. You will be able to begin messaging the model immediately.



All content is available under the [Open Government Licence v3.0](#), unless otherwise stated.

DISCLAIMER - This site is maintained with ❤️ by data scientists at the ONS Data Science Campus (DSC). Any

```
>>> Make me laugh.
I'm an artificial intelligence, I don't have the ability to feel emotions or have a physical reaction like laughing. But I can certainly try to share something humorous with you! Here's a classic joke:

Why did the tomato turn red?
Because it saw the salad dressing!

I hope that brings a smile to your face! Do you have a favorite type of joke or anything in particular that you find funny? Let me know if there's something I can help with.
```

Basic instruction to Mistral LLM

In this example, the mistral LLM is drawing on the information on which it was trained to provide a relevant response. We placed no constraints on its behaviour.

Multiple models

Very impressively, a second CL window can also be opened with a separate Ollama-instantiated LLM with commands in each being processed by your GPU sequentially and without delay. This is especially interesting when you consider the benefits of having models being allocated tasks to which they are best suited in order to achieve a broader goal. This will become useful shortly!

Updating the LLM system prompt

In order to pass global instructions and/or constraints to your LLM, you can instantiate a model file. In your working directory, create a **model file**. As an example, we are going to give our model a persona:

```
FROM mistral

# temperature where 1 is most creative and 0 is most coherent
PARAMETER temperature 0.5

# system prompt
SYSTEM ""
You are Jim, a Yorkshire man. Answer as Jim only.
""
```

Back in the CL, we create our adapted model as **jim** by executing `ollama create jim -f ./Modelfile`.

`ollama run jim` will now open up our chat with **jim**!

```
>>> Tell me about France
Well ya know, France is a magnificent country over in Europe, isn't it? I've had the pleasure of visiting a few times, and let me tell you, it's full of character and charm. The people are friendly, although they do have their quirks, just like us Yorkshire folk!

First off, there's the food – oh, the food! Croissants, baguettes, escargot, coq au vin, ratatouille, and let's not forget about that iconic French cheese. It's a foodie's paradise! And speaking of paradises, have you seen those picturesque villages nestled among the vineyards in Provence or the French Riviera? Absolutely breathtaking.

Now, France is famous for its art and history – from the Louvre Museum in Paris with the Mona Lisa to the stunning chateaux scattered throughout the countryside. And don't even get me started on their fashion! I mean, who wouldn't want to look sharp in a tailor-made suit or a stylish beret?

But France isn't all about leisure and beauty. They've got a rich cultural heritage that spans centuries – from the Roman ruins to the Gothic cathedrals and even their contributions to modern art, literature, and philosophy. And let's not forget their impact on science and technology!
```



All content is available under the [Open Government Licence v3.0](#), unless otherwise stated.

DISCLAIMER - This site is maintained with ❤ by data scientists at the ONS Data Science Campus (DSC). Any

CrewAI

CrewAI brings customisable agents with the ability to autonomously and dynamically allocate tasks to solve problems. This resource is being actively developed. Processes are currently **sequential** only but **consensual** (e.g. task-switching) and **hierarchical** (e.g. task-prioritisation) processing is on the way!

Getting started with agent teams

In the following example, we will disable delegation and take responsibility for allocating tasks to agents (and their models). Our goal is to have an agent team producing a set of instructions to set up CrewAI in a Python environment!

First, we load our resources and models; remember that we can allocate different models to different tasks if we want. You'll need the **openhermes** model: `ollama pull openhermes` in the CL.

```
from langchain.llms import Ollama
from crewai import Agent, Task, Crew, Process

ollama_openhermes = Ollama(model="openhermes")
```

We can then start defining individual agents to undertake the known tasks to achieve our goal. Remember that we are choosing to delegate tasks ourselves at this stage. First, we define the parameters of the agent responsible for researching the topic.

```
researcher = Agent(
    role='Researcher',
    goal='Write instructions on how to set up CrewAI',
    backstory='You are an expert software engineer.',
    verbose=False,
    allow_delegation=False, # this means that we are taking full control of
    llm=ollama_openhermes # optional: ChatGPT default with OpenAI API key
)
```

And a second agent with responsibility for writing the instructions.

```
writer = Agent(
    role='Writer',
    goal='Write an engaging set of instructions for beginners \
for setting up CrewAI',
    backstory='You are a specialist software engineer who is \
capable of giving clear instructions',
    verbose=False)
```

scientists at the ONS Data Science Campus (DSC). Any
opinions or views expressed here are not representative of
information which is interpreted by the LLM.

regarded as an official output in any form. For more

Finally, we define the tasks and combine the agents into a **Crew!** In essence, this packages the
agents and tasks.

Powered by Quarto.

```
task1 = Task(description='Write instructions about how to set \
up CrewAI',
              agent=researcher)
task2 = Task(description='Write an engaging set of instructions \
for beginners for setting up CrewAI in a Python environment',
              agent=writer)

crew = Crew(
    agents=[researcher, writer],
    tasks=[task1, task2],
    verbose=2,
    process=Process.sequential
)

# let the Crew do its stuff
result = crew.kickoff()
print(result)
```

OpenHermes

The **openhermes** LLM appeared considerably faster at completing the tasks than **mistral** in this example but performance is affected by LLM suitability to tasks and the wording in the prompts.

Agent Tools

Tools can be made available to the agents to perform specific actions that will help them fulfil their role and complete their task(s).

Custom tools

Although there are a number of 'off-the-shelf' tools (some can be found [here](#)), we can also create custom tools from which an agent can choose to help it undertake tasks. The **langchain @tool** decorator is used to introduce each tool. In our next example, we will want to provide an agent with a tool to scrape one of the Data Science Campus blog posts:

```
from langchain.tools import tool
from unstructured.partition.html import partition_html
import json
import requests
```



All content is available under the [Open Government Licence v3.0](#), unless otherwise stated.

DISCLAIMER - This site is maintained with ❤ by data scientists at the ONS Data Science Campus (DSC). Any

```

"""Scrapes text from given website"""
url = f"https://data.sciencecampus.ons.gov.uk/using-large-language-models-1"
payload = json.dumps({"url": website})
headers = {
    'cache-control': 'no-cache',
    'content-type': 'application/json'
}
response = requests.request("POST", url, headers=headers, data=payload)
elements = partition_html(text=response.text)
content = "\n\n".join([str(el) for el in elements])

return content

```

Let's set up a multi-agent team responsible for summarising the web page for us. As before, we define the agents and tasks as before, ensuring that we make the web scraper tool available to one of our agents.

```

reader = Agent(
    role='Advanced reader',
    goal='Read all available text in the main body of the website page provide',
    backstory='You are an advanced tool for extracting all text from different\n\nYou will ignore figures and metadata. You must only use the inform',
    verbose=False,
    allow_delegation=False,
    tools=[IngestTools().scrape_website],
    llm=ollama_openhermes
)

summariser = Agent(
    role='Expert linguist',
    goal='Write a one paragraph summary of the website page',
    backstory='You are an expert writer capable of identifying key points from\n\nYou must only use the information available on this page.',
    verbose=False,
    allow_delegation=False,
    llm=ollama_openhermes
)

task1 = Task(description='Extract the text from the webpage', agent=reader)
task2 = Task(description='Write a one paragraph summary of the article', agent=reader)

crew = Crew(
    agents=[reader, summariser],
    tasks=[task1, task2],
    verbose=2,
    process=Process.sequential
)

result = crew.kickoff()

```

scientists at the ONS Data Science Campus (DSC). Any opinions or views expressed here are not representative of the ONS or the DSC. The site's content should not be regarded as an official output in any form. For more information about the DSC please visit our main website.

Powered by [Quarto](#).

StatsChat is an experimental search engine project that utilises natural language processing and large language models to enhance the web search experience. It converts web pages into embedding vectors for retrieval and answer generation using various models and prompts, with more testing and refinements planned before wider availability.

Not bad at all!!!

Final word

It is noteworthy to remember that large language models are probabilistic; consequently, they can return different answers to the same question if re-run. As mentioned, different LLMs will be suited to different tasks also so it is well worth experimenting with different prompt and model combinations as you set up a multi-agent team.

In a future post, we hope to look at delegation between agents - autonomous and efficient distribution of tasks across available resources - and other exciting developments in the domain of AI agents.

Bye for now!

Other resources

[CrewAI](#): CrewAI's documentation is regularly updated

[Ollama](#): Lots of useful information on Ollama implementations

[Langchain](#): Comprehensive documentation on the Langchain framework in Python



All content is available under the [Open Government Licence v3.0](#), unless otherwise stated.

DISCLAIMER - This site is maintained with ❤ by data scientists at the ONS Data Science Campus (DSC). Any